

Tema 9 — Módulos

Laboratorio Guiado

Objetivos	Practicar la creación de módulos propios, las formas principales de importación, el uso de <code>__name__ == "__main__"</code> , el diagnóstico de errores de importación y la organización de código en paquetes.
Material necesario	Terminal, Python 3.13, venv del Tema09, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en <code>~/Curso_Python/Tema09/src</code> .

Laboratorio 1. Importar un módulo completo

1. Actualizar el repositorio, abrir la carpeta del tema y activar el entorno virtual. El venv ya está creado.

```
git pull origin main
cd ~/Curso_Python/Tema09/src
source ../.venv/bin/activate
python --version
code ..
```

2. Revisar el módulo `validaciones.py`. Este fichero contiene funciones reutilizables y no se ejecuta como script principal.

```
"""
Tema 9 - Módulos
Módulo de validaciones reutilizables.

Este fichero actúa como módulo propio. No está pensado para ejecutarse como
script principal, sino para ser importado desde otros ficheros.
"""

def puerto_valido(puerto: int) -> bool:
    """Devuelve True si el puerto está dentro del rango TCP/UDP válido."""
    return 1 <= puerto <= 65535

def normalizar_servicio(nombre: str) -> str:
    """Elimina espacios sobrantes y convierte el servicio a minúsculas."""
    return nombre.strip().lower()

def estado_valido(estado: str) -> bool:
    """Comprueba si el estado pertenece a los valores esperados."""
    return estado.strip().upper() in ("OK", "ERROR", "DETENIDO")
```

3. Abrir el fichero `01_import_completo.py`. Importar el módulo completo y usar sus funciones con la sintaxis `modulo.recurso`.

```
"""
Tema 9 - Módulos
Laboratorio 1: importar un módulo completo.
```

```
Objetivo:
    Importar el módulo validaciones completo y acceder a sus recursos usando
    la sintaxis modulo.recurso.
"""

import validaciones

puerto = 443

if validaciones.puerto_valido(puerto):
    print("Puerto correcto")

servicio = validaciones.normalizar_servicio(" SsH ")
print("Servicio normalizado:", servicio)

estado = " ok "
print("Estado válido:", validaciones.estado_valido(estado))
```

4. Ejecutar el script completo desde la terminal.

```
python 01_import_completo.py
```

Resultado esperado: el alumno debe comprender que import validaciones carga el módulo y obliga a usar el prefijo validaciones. para acceder a sus funciones.

Laboratorio 2. Importar recursos concretos

1. Abrir el fichero 02_import_recurso.py. Revisar cómo se importa una sola función desde un módulo.

```
"""
Tema 9 - Módulos
Laboratorio 2: importar recursos concretos.

Objetivo:
    Importar solo una función concreta desde un módulo.
"""

from validaciones import puerto_valido

if puerto_valido(22):
    print("ssh válido")

# Esta línea generaría NameError porque normalizar_servicio
# no se ha importado en este script.
# servicio = normalizar_servicio(" SsH ")
# print(servicio)
```

2. Comprobar que puerto_valido() queda disponible directamente, sin prefijo de módulo.

```
python 02_import_recurso.py
```

3. Revisar las líneas comentadas. Explican que normalizar_servicio() generaría NameError porque no fue importada en este script.

```
# servicio = normalizar_servicio(" SsH ")
```

```
# print(servicio)
```

Resultado esperado: el alumno debe diferenciar importar un módulo completo de importar solo un recurso concreto.

Laboratorio 3. Importar con alias

1. Abrir el fichero 03_import_alias.py. Revisar el uso de alias para un módulo completo y para una función concreta.

```
"""
Tema 9 - Módulos
Laboratorio 3: importar con alias.

Objetivo:
    Usar alias tanto para un módulo completo como para un recurso concreto.
"""

print("=== Alias de módulo completo ===")

import validaciones as val

if val.puerto_valido(443):
    print("https válido")

print("Servicio:", val.normalizar_servicio(" NGINX "))

print("\n=== Alias de recurso concreto ===")

from validaciones import puerto_valido as valido

if valido(22):
    print("ssh válido")

# Esta línea generaría NameError porque no se importó el nombre original.
# print(puerto_valido(443))
```

2. Ejecutar el script y comprobar que val.puerto_valido() procede del alias del módulo, mientras que valido() procede del alias de la función.

```
python 03_import_alias.py
```

3. Revisar la línea comentada final. El nombre original puerto_valido no queda disponible cuando se importa como valido.

```
# print(puerto_valido(443))
```

Resultado esperado: el alumno debe usar alias con criterio, entendiendo que mejoran legibilidad solo cuando siguen siendo claros.

Laboratorio 4. Importar todo con *

1. Abrir el fichero 04_import_todo_no_recomendado.py. Revisar que from validaciones import * incorpora los nombres públicos del módulo al espacio actual.

```
"""
Tema 9 - Módulos
Laboratorio 4: importar todo con *.

Objetivo:
    Mostrar que import * funciona, pero no es recomendable en código
    profesional porque oculta el origen de los nombres importados.
"""

# Evitar esta forma salvo casos muy justificados.
from validaciones import *

puerto = 443

if puerto_valido(puerto):
    print("Puerto correcto")

servicio = normalizar_servicio(" SsH ")
print("Servicio normalizado:", servicio)

estado = " error "
print("Estado válido:", estado_valido(estado))
```

2. Ejecutar el script completo.

```
python 04_import_todo_no_recomendado.py
```

3. Analizar por qué esta forma funciona, pero no es recomendable en código profesional: oculta el origen de las funciones y puede provocar colisiones de nombres.

Resultado esperado: el alumno debe reconocer import * como una práctica a evitar salvo casos muy justificados.

Laboratorio 5. Módulo importable frente a script ejecutable

1. Abrir diagnostico.py. Revisar las funciones reutilizables y el bloque if __name__ == "__main__".

```
"""
Tema 9 - Módulos
Módulo de diagnóstico.

Este ejemplo documenta el uso de if __name__ == "__main__".
"""

def comprobar() -> str:
    """Devuelve un mensaje sencillo de diagnóstico."""
    return "sistema operativo OK"

def obtener_estado_servicios() -> list[dict]:
    """Devuelve una lista simulada de servicios."""
    return [
        {"nombre": "ssh", "estado": "OK"},
        {"nombre": "nginx", "estado": "OK"},
        {"nombre": "api", "estado": "ERROR"},
    ]
```

```
def resumen_servicios(servicios: list[dict]) -> str:
    """Genera un resumen simple de servicios."""
    total = len(servicios)
    errores = 0

    for servicio in servicios:
        if servicio["estado"] == "ERROR":
            errores += 1

    return f"Servicios revisados: {total}. Servicios con error: {errores}"

# Este bloque solo se ejecuta si lanzamos:
# python diagnostico.py
#
# No se ejecuta cuando otro script hace import diagnostico.
if __name__ == "__main__":
    print("Iniciando diagnóstico")
    print(comprobar())

    servicios = obtener_estado_servicios()
    print(resumen_servicios(servicios))
```

2. Ejecutar diagnostico.py directamente. En este caso sí se ejecuta el bloque `__main__`.

```
python diagnostico.py
```

3. Abrir `05_main_diagnostico.py`. Importar `diagnostico` como módulo y usar sus funciones. El bloque `__main__` de `diagnostico.py` no debe ejecutarse al importarlo.

```
"""
Tema 9 - Módulos
Laboratorio 5: importar un módulo con bloque __main__.

Objetivo:
Comprobar que el bloque if __name__ == "__main__"
no se ejecuta cuando el fichero se importa como módulo.
"""

import diagnostico

print("Comprobación:", diagnostico.comprobar())

servicios = diagnostico.obtener_estado_servicios()
print("Servicios:", servicios)

resumen = diagnostico.resumen_servicios(servicios)
print("Resumen:", resumen)
```

4. Ejecutar el script principal.

```
python 05_main_diagnostico.py
```

Resultado esperado: el alumno debe distinguir entre ejecutar un fichero directamente e importarlo como módulo reutilizable.

Laboratorio 6. Errores frecuentes al importar

1. Abrir el fichero 06_errores_importacion.py. Revisar cómo se controlan ImportError y ModuleNotFoundError sin romper el laboratorio.

```
"""
Tema 9 - Módulos
Laboratorio 6: errores frecuentes al importar.

Objetivo:
    Identificar errores comunes de importación sin romper el laboratorio.
"""

print("=== Error 1: recurso inexistente ===")

try:
    from validaciones import validar_ip
except ImportError as error:
    print("ImportError controlado:")
    print(error)

print("\n=== Error 2: módulo inexistente ===")

try:
    import modulo_que_no_existe
except ModuleNotFoundError as error:
    print("ModuleNotFoundError controlado:")
    print(error)

print("\n=== Diagnóstico recomendado ===")
print("- Comprobar el nombre del fichero .py")
print("- Comprobar que estamos ejecutando desde el directorio correcto")
print("- Revisar si el recurso existe dentro del módulo")
print("- Evitar nombres de fichero como random.py, json.py o sys.py")
print("- Revisar si dos módulos se importan entre sí")
```

2. Ejecutar el script y revisar los mensajes de diagnóstico.

```
python 06_errores_importacion.py
```

3. Comprobar después de varias ejecuciones que puede aparecer el directorio __pycache__. Es normal y puede eliminarse sin afectar al código fuente.

```
ls -la
tree __pycache__
```

Resultado esperado: el alumno debe identificar errores de importación habituales y aplicar una lista breve de comprobaciones.

Laboratorio 7. Ejemplo integrado de paquete propio

1. Revisar la estructura del paquete soporte_ti dentro del directorio src. El paquete agrupa módulos relacionados bajo un mismo directorio.

```
src/
├── 07_paquete_soporte_ti_demo.py
└── soporte_ti/
```

```
| _init__.py  
| red.py  
| reportes.py
```

2. Abrir soporte_ti/_init__.py. Este fichero identifica el paquete y define metadatos básicos como la versión.

```
"""  
Paquete soporte_ti.  
  
Agrupar módulos sencillos para operaciones típicas de soporte:  
validación de red y generación de reportes.  
"""  
  
__version__ = "1.0.0"
```

3. Abrir soporte_ti/red.py. Revisar las funciones de normalización y validación de servicios de red.

```
"""  
Módulo red del paquete soporte_ti.  
  
Contiene funciones de normalización y validación de servicios de red.  
"""  
  
def normalizar_servicio(nombre: str) -> str:  
    """Normaliza nombres de servicio para comparación y reporte."""  
    return nombre.strip().lower()  
  
def validar_puerto(puerto: int) -> bool:  
    """Devuelve True si el puerto está dentro del rango TCP/UDP válido."""  
    return 1 <= puerto <= 65535  
  
def validar_ip_simple(ip: str) -> bool:  
    """  
    Valida una dirección IPv4 de forma básica.  
  
    Reglas:  
    - Debe tener cuatro partes separadas por puntos.  
    - Cada parte debe ser numérica.  
    - Cada número debe estar entre 0 y 255.  
    """  
    partes = ip.split(".")  
  
    if len(partes) != 4:  
        return False  
  
    for parte in partes:  
        if not parte.isdigit():  
            return False  
  
        numero = int(parte)  
        if numero < 0 or numero > 255:  
            return False  
  
    return True
```

```
def comprobar_servicio(servicio: dict) -> dict:
    """Normaliza y valida un registro de servicio."""
    nombre = normalizar_servicio(servicio["nombre"])
    ip = servicio["ip"]
    puerto = servicio["puerto"]

    return {
        "nombre": nombre,
        "ip": ip,
        "puerto": puerto,
        "ip_valida": validar_ip_simple(ip),
        "puerto_valido": validar_puerto(puerto),
    }

def comprobar(servicios: list[dict]) -> list[dict]:
    """Comprueba una lista de servicios y devuelve resultados normalizados."""
    resultados = []

    for servicio in servicios:
        resultado = comprobar_servicio(servicio)
        resultados.append(resultado)

    return resultados
```

4. Abrir soporte_ti/reportes.py. Revisar cómo se genera una salida legible a partir de resultados procesados.

```
"""
Módulo reportes del paquete soporte_ti.

Genera salidas legibles a partir de datos procesados por otros módulos.
"""

def generar_linea(resultado: dict) -> str:
    """Genera una línea de reporte para un resultado de comprobación."""
    estado_ip = "IP_OK" if resultado["ip_valida"] else "IP_ERROR"
    estado_puerto = "PUERTO_OK" if resultado["puerto_valido"] else "PUERTO_ERROR"

    return (
        f"{resultado['nombre']:<12} "
        f"{resultado['ip']:<15} "
        f"{resultado['puerto']:<6} "
        f"{estado_ip:<8} "
        f"{estado_puerto}"
    )

def generar_resumen(resultados: list[dict]) -> str:
    """Genera un resumen completo a partir de resultados validados."""
    lineas = [
        "SERVICIO      IP                      PUERTO ESTADO_IP ESTADO_PUERTO",
        "-" * 62,
    ]
```



```
for resultado in resultados:
    lineas.append(generar_linea(resultado))

return "\n".join(lineas)
```

5. Abrir 07_paquete_soporte_ti_demo.py. Revisar cómo se importan los módulos del paquete y cómo se aplica el flujo completo.

```
"""
Tema 9 - Módulos
Laboratorio 7: ejemplo integrado de paquete propio.

Estructura:
Tema09/
└─ src/
    └─ 07_paquete_soporte_ti_demo.py
        └─ soporte_ti/
            ├── __init__.py
            ├── red.py
            └── reportes.py

Objetivo:
    Usar un paquete propio con varios módulos internos.
"""

from soporte_ti import __version__
from soporte_ti import red
from soporte_ti import reportes

def main():
    """Ejecuta el ejemplo integrado del paquete soporte_ti."""
    print("=== 1. Información del paquete ===")
    print("Paquete soporte_ti versión:", __version__)

    print("\n=== 2. Datos de servicios ===")
    servicios = [
        {"nombre": "SSH", "ip": "10.0.0.10", "puerto": 22},
        {"nombre": "NGINX", "ip": "10.0.0.20", "puerto": 443},
        {"nombre": "api", "ip": "ip_invalida", "puerto": 8080},
        {"nombre": "backup", "ip": "10.0.0.40", "puerto": 70000},
    ]

    for servicio in servicios:
        print(servicio)

    print("\n=== 3. Comprobar servicios con soporte_ti.red ===")
    resultados = red.comprobar(servicios)

    for resultado in resultados:
        print(resultado)

    print("\n=== 4. Generar resumen con soporte_ti.reportes ===")
    resumen = reportes.generar_resumen(resultados)
    print(resumen)

if __name__ == "__main__":
    main()
```

6. Ejecutar el ejemplo integrado desde el directorio src.

```
python 07_paquete_soporte_ti_demo.py
```

Resultado esperado: el alumno debe comprender cómo organizar un paquete pequeño, importar sus módulos y reutilizar sus funciones desde un script principal.

Ejercicios propuestos

1. Crear un script src/08_inventario_modular.py que importe validaciones.py y procese una lista de servicios con nombre, puerto y estado.
2. Crear un módulo inventario.py con una función cargar_servicios() que devuelva una lista de diccionarios simulando un inventario de servicios.
3. Crear un módulo reportes.py con una función generar_resumen(servicios) que cuente cuántos servicios tienen estado OK, ERROR o DETENIDO.
4. Crear un script main_inventario.py que importe inventario.py, validaciones.py y reportes.py para generar un resumen completo.
5. Crear un script probar_name_main.py que importe diagnostico.py y después ejecutar diagnostico.py directamente para comparar el comportamiento.
6. Añadir al paquete soporte_ti un módulo usuarios.py con una función normalizar_usuario(nombre) y usarla desde un nuevo script de prueba.
7. Provocar de forma controlada un ModuleNotFoundError cambiando temporalmente el nombre de un import, y documentar el diagnóstico realizado.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Tema09) o el venv no está activo	El entorno virtual no se ha activado desde el directorio correcto.	Ejecutar <code>cd ~/Curso_Python/Tema09/src</code> y <code>source ../.venv/bin/activate</code> .
ModuleNotFoundError al importar validaciones	Python no encuentra el módulo porque el script se ejecuta desde otro directorio o el fichero no está en src.	Ejecutar el script desde <code>~/Curso_Python/Tema09/src</code> y comprobar <code>ls -la validaciones.py</code> .
ImportError al importar una función concreta	El módulo existe, pero el recurso solicitado no está definido o tiene otro nombre.	Abrir el módulo y comprobar el nombre exacto de la función, clase o constante.
NameError al usar <code>normalizar_servicio()</code>	La función no se importó directamente o se importó con alias.	Usar <code>validaciones.normalizar_servicio()</code> , importarla explícitamente o usar el

		alias correcto.
El bloque <code>__main__</code> se ejecuta cuando no debería	Hay código suelto fuera de <code>if __name__ == "__main__":</code> .	Mover las pruebas o la ejecución principal dentro de una función <code>main()</code> y protegerla con el bloque <code>__main__</code> .
Aparece el directorio <code>__pycache__</code>	Python ha compilado módulos importados a bytecode para optimizar la carga.	Es normal. No versionarlo en Git y eliminarlo si se quiere limpiar el directorio.
El paquete <code>soporte_ti</code> no se importa	El script no se ejecuta desde el directorio que contiene el paquete o falta la carpeta <code>soporte_ti</code> .	Comprobar <code>pwd</code> , ejecutar desde <code>src</code> y revisar que existe <code>soporte_ti/__init__.py</code> .
Error al usar <code>from .red import ...</code> fuera del paquete	Los imports relativos solo funcionan dentro de módulos que forman parte de un paquete.	Usar import absoluto desde el script principal: <code>from soporte_ti import red</code> .
Colisión con módulos estándar	Un fichero local se llama igual que un módulo de Python, por ejemplo <code>json.py</code> o <code>random.py</code> .	Renombrar el fichero local y borrar <code>__pycache__</code> si queda bytecode antiguo.
El código funciona en VS Code pero no en terminal	La terminal está en otro directorio o usa otro intérprete.	Comprobar <code>pwd</code> , <code>python --version</code> y el entorno seleccionado en VS Code.